



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



**FACULTAD
DE INGENIERÍA**

Control de Posición de Motor de CC

Sistemas de Automatización

Integrantes:

Martinez Julio **Leg:**14733

Sanchez Ignacio **Leg:**14616

Scarafia Valentino **Leg:**14734

Sicoli Giuliano **Leg:**14604

Índice

1. Introducción	1
1.1. Decodificador de posición angular	1
1.2. Caja Reductora	3
2. Desarrollo del Sistema de Control	3
2.1. Obtención de Ecuaciones para el Modelado	3
2.2. Obtención de la Función de Transferencia	4
3. Simulación y Resultados	6
3.1. Código	6
3.2. Resultados	9
4. Conclusión	12
5. Fuentes	12

1. Introducción

La problemática surge con la necesidad de controlar la posición angular del eje del motor a través de la utilización de un control de lazo realimentado con algoritmo PID para una posible aplicación mecatrónica o industrial a definir. Ya que este, dependiendo de su aplicación, podría ser el encargado de la cinemática de un brazo robótico o máquina para una cinta transportadora. En el proyecto se utilizó como modelo un motor Maxon RE25-20W de corriente continua con una capacidad de voltaje de 24V. Esto sirvió para la utilización de las constantes reales de un motor físico buscadas en su respectivo datasheet.

Cuadro 1: Parámetros del motor DC (Maxon RE25)

Parámetro	Símbolo	Valor	Unidad
Resistencia	R	1,53	Ω
Constante de torque	K_t	0,0208	N·m/A
Constante electromotriz	K_b	0,0208	V·s/rad
Inercia del rotor	J	$1,47 \times 10^{-6}$	kg·m ²
Fricción viscosa	B	$3,0 \times 10^{-6}$	N·m·s/rad

1.1. Decodificador de posición angular

Para lograr convertir la posición angular del eje a una señal se requiere un decodificador. Para esta simulación se eligió E6B2-CWZ6C 1024P/R el cual se encarga de transformar la rotación del eje a una señal eléctrica de voltaje, la cual será usada para calcular el error y corregirlo a través de la retroalimentación.

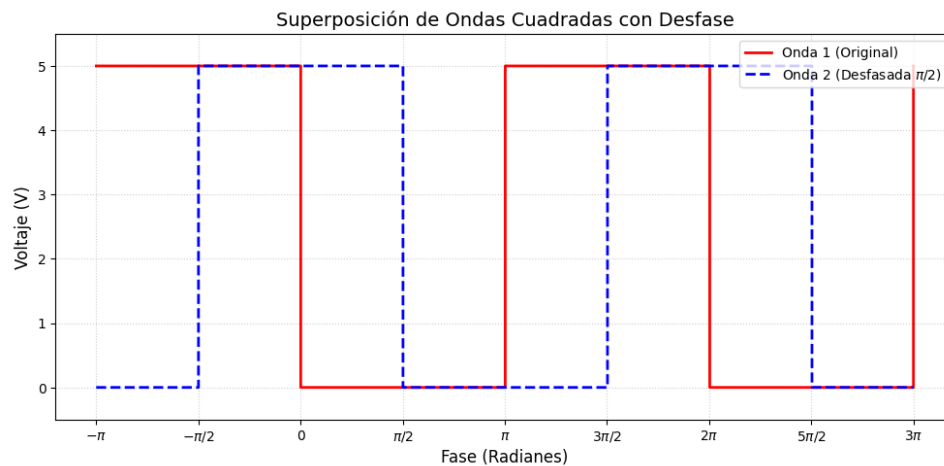


Figura 1: Pulsos del Decodificador

Este sensor no basa su funcionamiento, a diferencia de un potenciómetro rotativo, en la determinación de la posición angular absoluta de su eje, sino que trabaja generando pulsos y sumándolos, es así como se puede obtener la diferencia entre la posición final e inicial.

Internamente posee un disco ranurado ópticamente. Al girar el eje, unos sensores ópticos detectan el paso de la luz a través de las ranuras y generan ondas cuadradas en las salidas. Estas ondas se encuentran desfasadas 90° grados eléctricos las cuales permiten detectar si se está rotando en una posición antihorario u horario, estas están representadas en la Figura 1. Por ejemplo para un punto medio del pulso de la onda roja, si hay desplazamiento hacia un lado habrá una señal neutra y hacia el otro lado habrá una señal positiva, todo esto para una misma señal positiva de la onda roja.

Simulación Interna: Sensor Óptico de Ranura

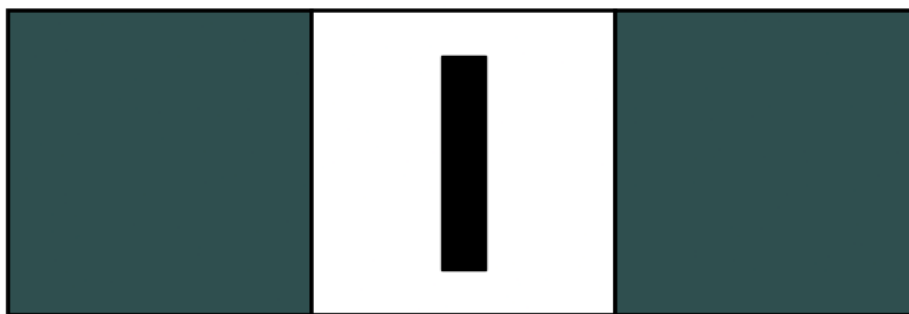


Figura 2: Simulación Interna Sensor Óptico

Se aprecia una simplificación de su estructura interna, siendo el rectángulo negro una simulación del sensor óptico que al pasar por la luz (cuadrado blanco) emite un pulso hasta que queda obstruido por el cuadrado siguiente.

El sensor posee, como su nombre indica 1024 ranuras por revolución y por su forma de construcción es posible conseguir medir 4096 ranuras por revolución. Al dividir dicho valor en /2 para convertir las unidades a ranuras por revolución, se obtiene la constante del encoder que

se utilizará en el controlador PID.

$$\frac{4096 \text{ pulsos}}{\text{rev}} \times \frac{1 \text{ rev}}{2\pi \text{ rad}} = 651,9 \frac{\text{pulsos}}{\text{rad}}$$

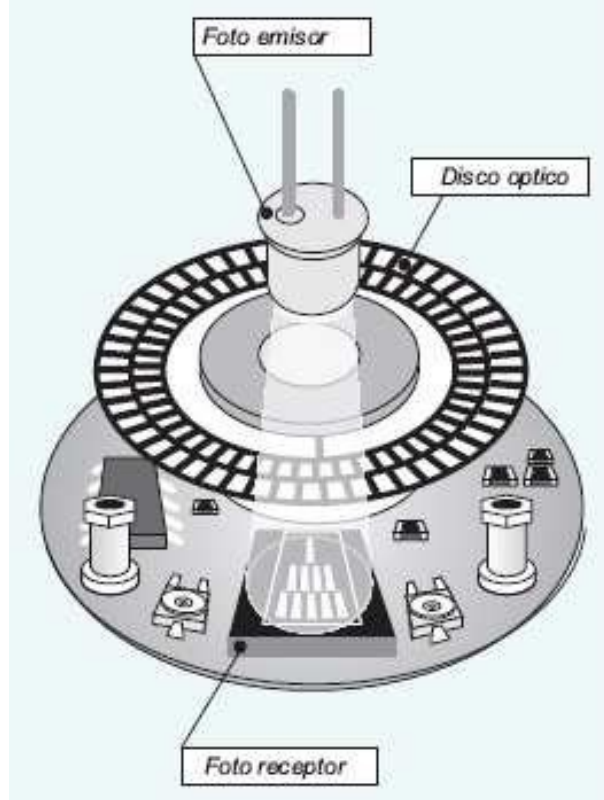


Figura 3: Simulación Interna Sensor Óptico

1.2. Caja Reductora

Se optó por la utilización de una caja reductora ya que un motor de DC tiene usualmente una muy buena velocidad pero un torque reducido lo cual es compensado por la caja reductora. La caja consta de dos engranajes acoplados con una relación de N a 1, significando que el motor debe girar N vueltas para que el eje de la caja reductora gire 1, además la caja provee de una ganancia de N veces el torque del motor. En el desarrollo de este trabajo se utilizó $N = 10$. El momento de inercia entonces no es solo el del rotor, además de este se encuentra el de la caja reductora. Entonces el momento de inercia total que tendrá que acelerar el motor es:

$$J_{Total} = J_{Motor} + J_{Caja} + \frac{J_{Carga}}{N^2}$$

2. Desarrollo del Sistema de Control

2.1. Obtención de Ecuaciones para el Modelado

Para modelar este sistema se tendrá en cuenta su aspecto eléctrico y dinámico. Para obtener las ecuaciones que modelan el comportamiento eléctrico se partirá de la ley de Kirchhoff aplicada sobre el circuito del motor:

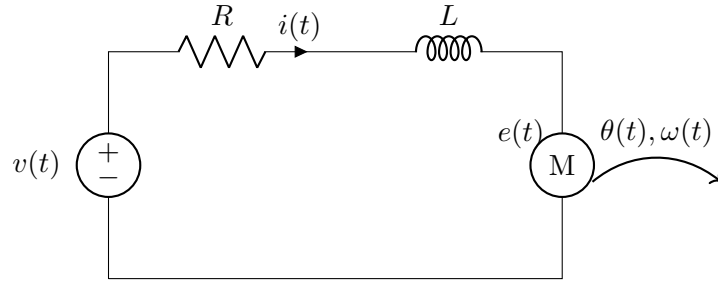


Figura 4: Esquema electromecánico equivalente del motor de corriente continua.

Aplicando ley de mallas:

$$v(t) - Ri(t) - L \frac{di(t)}{dt} - e(t) = 0$$

$$v(t) = Ri(t) + L \frac{di(t)}{dt} + e(t)$$

Y la fuerza contraelectromotriz $e(t)$ se relaciona proporcionalmente con la velocidad angular del eje del motor, lo que da la siguiente ecuación:

$$e(t) = k_e \omega$$

Para el modelado dinámico se partió de la ley de Newton para la rotación:

$$\sum \tau = J\ddot{\theta}$$

Como no hay ningún elemento dentro del motor que tenga un comportamiento elástico las únicas dos fuerzas en juego son la fricción viscosa y el momento aplicado por la corriente circulante en el motor.

$$J\ddot{\theta} = -B\dot{\theta} + T_m(t)$$

El torque proporcionado por el motor $T_m(t)$ depende de la corriente que circule por el motor, por ende:

$$T_m(t) = k_t i(t)$$

2.2. Obtención de la Función de Transferencia

Para el desarrollo del sistema de control primero definiremos las ecuaciones que rigen nuestro sistema. Estas ecuaciones son:

$$v(t) = Ri(t) + L \frac{di(t)}{dt} + e(t) \quad (1)$$

$$e(t) = k_e \dot{\theta} \quad (2)$$

$$J\ddot{\theta} = -B\dot{\theta} + T_m(t) \quad (3)$$

$$T_m(t) = k_t i(t) \quad (4)$$

Dado que los valores de la inductancia son mínimos, se despreciarán sus efectos. Reemplazando (4) en (3), y aplicando transformada de Laplace en (3) se obtuvo la siguiente relación entre corriente y posición angular:

$$I(s) = \Theta(s) \frac{Js^2 + Bs}{k_t} \quad (5)$$

También reemplazando (2) en (1), y aplicando transformada de Laplace en (1), se obtiene:

$$V(s) = I(s)R + k_e s \Theta(s) \quad (6)$$

Ahora reemplazando (5) en (6) y trabajando matemáticamente se puede llegar a la expresión de la función de transferencia a lazo abierto, sin controlador y sin realimentación $G(s)$, la cual tiene la forma:

$$G(s) = \frac{\Theta(s)}{V(s)} = \frac{k_t}{RJs^2 + (k_e k_t + RB)s} \quad (7)$$

Ahora se define el controlador $C(s)$, el cual es un PID, por lo tanto su función de transferencia será:

$$C(s) = \frac{k_d s^2 + k_p s + k_i}{s} \quad (8)$$

Teniendo en cuenta que la realimentación se lleva a cabo mediante un decodificador, esta no es unitaria y es igual a una constante K_{enc} , esta constante tiene unidades de [Cuentas/rad]. Teniendo esto en cuenta se armó el diagrama de bloques del sistema: Ahora se procede a calcular

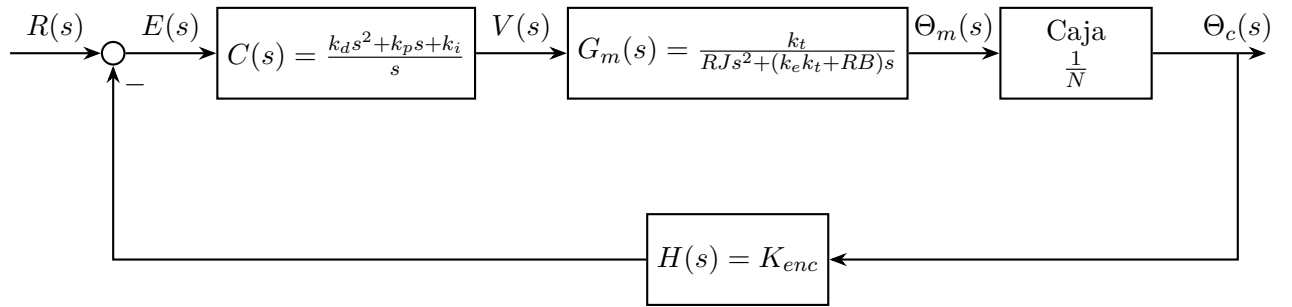


Figura 5: Diagrama en bloques del sistema de control de posición linealizado.

la función a lazo cerrado:

$$T(s) = \frac{G(s)C(s)(1/N)}{1 + G(s)C(s)(1/N)K_{enc}}$$

Trabajando esa expresión se obtuvo la función de transferencia a lazo cerrado que es la siguiente:

$$T(s) = \frac{k_t}{N} \frac{k_d s^2 + k_p s + k_i}{RJs^3 + (k_t k_e + RB + \frac{k_{enc} k_t k_d}{N})s^2 + \frac{k_{enc} k_t k_p}{N}s + \frac{k_{enc} k_t k_i}{N}} \quad (9)$$

Para el diseño analítico del controlador y la obtención de la función de transferencia a lazo cerrado, se asumirá una realimentación unitaria ideal trabajando en el dominio físico continuo (radianes). Esto permitirá trabajar en radianes la posición a la vez que se calculan los polos de manera más limpia. Para obtener valores de las constantes del controlador primero se eligieron los polos que se desea que tenga la ecuación característica, ahora se enfrenta la decisión de elegir 3 polos reales o dos complejos conjugados y uno real. La decisión más óptima es la de elegir dos polos complejos conjugados y uno real dado que tener 3 polos reales correspondería a una señal de respuesta sobreamortiguada lo que hace al sistema inherentemente lento. Mientras que con dos polos complejos se puede lograr que el sistema sea subamortiguado, logrando un tiempo de respuesta mucho más rápido pero con posibles oscilaciones. El tercer polo, el polo real se elige de tal manera que decaiga rápidamente su transitorio, esto permite que se defina el comportamiento del sistema casi exclusivamente con los dos polos complejos conjugados. Estos polos complejos dependen de parámetros que el diseñador determina, como el amortiguamiento y la frecuencia

natural que depende del tiempo de asentamiento deseado. Para elegir los polos complejos conjugados se eligió un valor de amortiguamiento $\zeta = 0,707$ y un tiempo de asentamiento $t = 1,0s$ lo que resulta en una frecuencia natural $\omega_n = \frac{4}{\zeta t} = 5,6577 \frac{rads}{s}$. Con estos datos se puede obtener la ecuación que representa estos dos polos complejos, la cual es:

$$s^2 + 2\zeta\omega_n s + \omega_n^2$$

Calculando las raíces del polinomio se obtienen los siguientes polos complejos conjugados: $p_1 = -4 + 4i$ y $p_2 = -4 - 4i$. El polo real elegido usualmente suele ser elegido como 5 o 15 veces la parte real del polo complejo, sin embargo, durante el proceso de simulación el polo $p_3 = -50$ no otorgaba la rapidez de convergencia deseada, por ende se terminó eligiendo colocar el polo en un valor más negativo para así asegurar su rápida convergencia. Este desacople entre la teoría y la simulación se debe a los ceros añadidos por el controlador, si estos quedan cerca del eje imaginario pueden cancelar parcialmente el efecto de un polo o generar un sobrepico o lentitud adicional impredecibles. Es culpa de estos que se debió reelegir el valor de la amortiguación ζ y el del tiempo de establecimiento para así alcanzar los objetivos elegidos previamente. Mediante pruebas se eligieron los valores $\zeta = 1,2$ y $t_s = 0,5$, con esto se tiene que $\omega_n = \frac{2}{3}$. Con estos valores y con el polo real elegido en $p_3 = -500$ es que se calculan las ganancias.

Una vez elegidos estos parámetros, se desarrolló la ecuación característica deseada, la cual es:

$$(s + p_3)(s^2 + 2\zeta\omega_n s + \omega_n^2) \\ s^3 + (2\zeta\omega_n + p_3)s^2 + (\omega_n^2 + 2\zeta\omega_n p_3)s + p_3\omega_n^2$$

Luego se la comparó con el denominador de la función de transferencia obtenida en (9), obteniendo así las expresiones de las constantes del controlador:

$$k_p = RJ \frac{2p_3\zeta\omega_n + \omega_n^2}{k_t/N} = 17,1602 \quad (10)$$

$$k_d = \frac{RJ[2\zeta\omega_n + p_3] - k_t k_e - RB}{k_t/N} = 0,8905 \quad (11)$$

$$k_i = RJ \frac{p_3\omega_n^2}{k_t/N} = 47,4038 \quad (12)$$

Cabe destacar que no se tuvo en cuenta en el cálculo de estas constantes el factor k_{enc} , esto se debe a que la simulación, internamente, convierte los valores utilizando esta constante, de esta manera se puede trabajar en radianes dentro del código. Además la señal del error será calculada en $\frac{volt}{rads}$ lo cual es más universal que trabajar en $\frac{volt}{pulso}$. De esta manera para reemplazar el encoder solo se tendrá que cambiar la constante de este en el código. De no ser así se debería recalcular las constantes del PID cuando cambiemos de encoder.

3. Simulación y Resultados

Una vez armado el controlador se probó la funcionalidad del sistema de control frente a distintas entradas para probar así si había habido algún error en el cálculo de las constantes o en la definición de parámetros como amortiguamiento o tiempo de asentamiento.

3.1. Código

Para la simulación se utilizó Python, el código se separó en dos partes, cálculo de las constantes PID y simulación. Como ya fueron tratadas analíticamente las constantes en la sección anterior ahora nos enfocaremos exclusivamente en la simulación.

Listing 1: Código de simulación en MATLAB

```

1      a1      = R * J;
2      a0      = (R * B) + k^2;
3      dt      = 0.0001;
4      tiempo  = 0:dt:12;
5      N      = length(tiempo);
6      U_fisica = pi * ones(1, N); % Referencia a 180 grados (pi rad)
7      voltaje  = zeros(1, N);
8      carga   = zeros(1, N);
9      % Vector de Estado: Fila 1 = Posición (theta), Fila 2 = Velocidad (
      omega)
10     x = zeros(2, N);
11     error_previo = 0;
12     integral     = 0;
13     % Matrices del Espacio de Estados (dx/dt = A*x + B*u)
14     A = [0, 1;
15          0, -a0/a1];
16     B = [0;
17          k/a1];
18     % ----- Bucle Principal (RK2) -----
19     for i = 2:N
20         % 1. Leer el estado anterior de la planta (instante i-1)
21         theta_ant = x(1, i-1);
22         omega_ant = x(2, i-1);
23         % 2. Transducción de posición (Simulación del encoder)
24         pos_carga_ant = theta_ant / N_c;
25         pulsos_rec     = pos_carga_ant * kenc;
26         pos_micro      = pulsos_rec / kenc;
27         % 3. Algoritmo del Controlador PID
28         error = U_fisica(i) - pos_micro;
29         proporcional = error * kp;
30         integral     = integral + (ki * error * dt);
31         derivativo   = ((error - error_previo) * kd) / dt;
32         V_pid = proporcional + integral + derivativo;
33         error_previo = error;
34         % 4. Saturación del voltaje de la fuente (+/- 24V)
35         if V_pid > 24
36             V_pid = 24;
37         elseif V_pid < -24
38             V_pid = -24;
39         end
40         voltaje(i) = V_pid;
41         % 5. Runge-Kutta 2
42         x_actual = x(:, i-1);
43         % Pendiente inicial (k1)
44         k1 = A * x_actual + B * V_pid;
45         % Predicción del estado futuro
46         x_estimado = x_actual + dt * k1;
47         % Pendiente en el extremo futuro (k2)
48         k2 = A * x_estimado + B * V_pid;
49         % Corrección del estado actual (Promedio ponderado) w=0.5
50         x(:, i) = x_actual + (dt / 2) * (k1 + k2);
51         % 6. Paso del giro del eje del motor al giro de la caja
52         carga(i) = x(1, i) / N_c;
53     end

```

En las líneas 25 y 26 puede observarse el porqué de no usar k_{enc} en el cálculo de las constantes

PID, dado que eventualmente trabajando de esta manera se puede obtener la posición que ve el microcontrolador de la siguiente manera:

$$posicion_medida_micro = \frac{posicion_carga \cdot k_{enc}}{k_{enc}} = posicion_carga$$

Normalmente esto no se programa en el microcontrolador y se pasa directo de $posicion_carga$ al cálculo del error, en esta ocasión se mantuvo en el código para fines explicativos.

Otro aspecto a destacar es el método de la simulación, para este se optó por un Runge-Kutta de orden 2. A continuación se detalla la obtención del modelo para la resolución por el anteriormente mencionado método:

Volviendo sobre las expresiones (1), (2), (3) y (4) se puede tomar la ecuación (1)

$$v(t) = Ri(t) + e(t)$$

y reemplazar la fuerza contraelectromotriz

$$v(t) = Ri(t) + k_e \omega(t)$$

de esta expresión se puede despejar $i(t)$.

$$i(t) = \frac{v(t) - k_e \omega(t)}{R}$$

Ahora se puede pasar a la parte dinámica reemplazando $i(t)$ en la expresión de la ley de Newton angular.

$$J\dot{\omega}(t) = -B\omega(t) + k_t \left(\frac{v(t) - k_e \omega(t)}{R} \right)$$

Multiplicando ambos lados por R

$$RJ\dot{\omega}(t) = -RB\omega(t) + k_t v(t) - k_t k_e \omega(t)$$

y reordenando:

$$RJ\dot{\omega}(t) = k_t v(t) - (RB + k_t k_e) \omega(t)$$

como $k_t = k_e = k$ y definiendo $a_1 = RJ$ y $a_0 = RB + k^2$ se obtiene la expresión

$$a_1 \dot{\omega}(t) = k v(t) - a_0 \omega(t)$$

de la cual se puede obtener la aceleración

$$\dot{\omega}(t) = \frac{k v(t) - a_0 \omega(t)}{a_1}$$

Ahora se debe reducir el orden de la ecuación, para ello se realizó la asignación:

$$\begin{aligned} \frac{d\theta}{dt} &= \omega(t) \\ \frac{d\omega}{dt} &= -\frac{a_0}{a_1} \omega(t) + \frac{k}{a_1} v(t) \end{aligned}$$

Que matricialmente quedaría:

$$\begin{bmatrix} \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{a_0}{a_1} \end{bmatrix} \begin{bmatrix} \theta \\ \omega \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{k}{a_1} \end{bmatrix} v(t) \quad (13)$$

Que aplicando el método Runge-Kutta se puede calcular:

$$x_i = x_{i-1} + \frac{dt}{2} (k_1 + k_2)$$

Con k_1 y k_2 calculados de la siguiente manera:

$$\begin{aligned} k_1 &= Ax_{i-1} + Bv_i \\ x_{aux} &= x_{i-1} + k_1 dt \\ k_2 &= A(x_{aux} + k_1 dt) + Bv_i \end{aligned}$$

3.2. Resultados

Primero se probó al sistema frente a una entrada tipo escalón, luego una entrada tipo rampa y después una entrada mixta, para ver cómo respondía a condiciones más extremas y entradas que varían más rápido; esto es para ver si es que el sistema sufre por el ajuste del control integral, derivativo o ambos. Para la respuesta a un escalón se le pide al motor que vaya hacia la posición de $180^\circ = \pi$ rads. Frente a este escalón el sistema tiene el siguiente comportamiento.

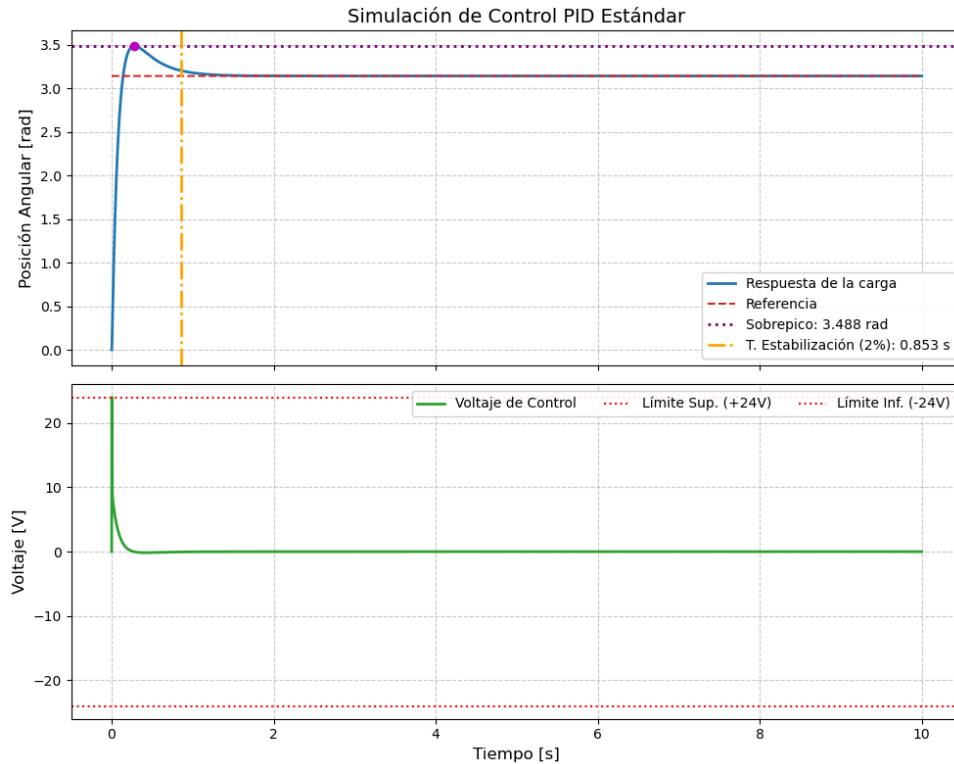


Figura 6: Respuesta del sistema frente a un escalón.

Cuadro 2: Métricas de Desempeño

Entrada	t_s (s)	M_p (%)	e_{ss} (rad)	V_{max} (V)
Escalón	0.853	11.020	0.000	24.000

Como se puede ver en la Figura 6 y en la Tabla 2, el sistema se estabiliza dentro de lo deseado, es decir en menos de un segundo. Sin embargo, tiene un sobrepico de unos $0,26 \text{ rads} = 15^\circ$ que decae rápidamente; esto muestra un comportamiento dentro de lo esperado ya que el sobrepico es del 11 %. Si bien para un valor de $\zeta = 0,707$ se espera un overshoot teórico de 4,3 %, debido a los ceros añadidos al sistema el comportamiento de este se modifica haciendo que los sobrepicos sean mayores, es por esto que se eligió un $\zeta = 1,2$. Sin embargo, el tiempo de estabilización se da a los $0,853 \text{ s}$ lo que indica que el tiempo de estabilización al 2 % se cumple. Luego se utilizó como entrada una recta.

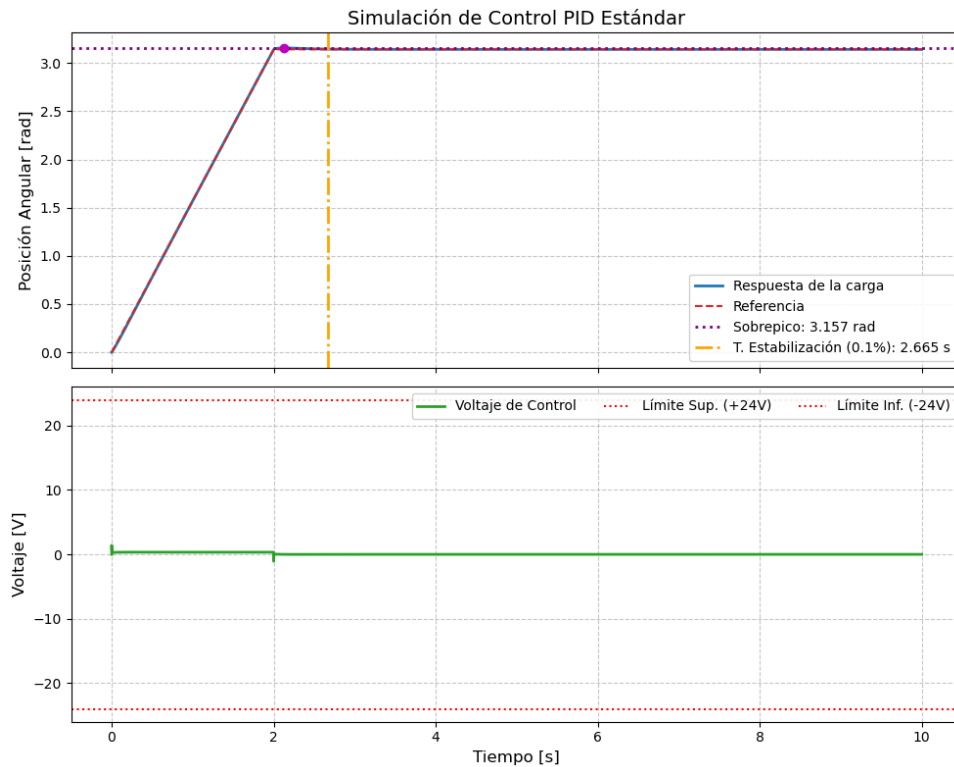


Figura 7: Respuesta del sistema frente a una recta.

Como se puede ver en la Figura 7, el sistema no se desprende nunca de la referencia durante los 2s donde se da el cambio abrupto en la misma. En este caso el tiempo de estabilización es casi nulo, así que se lo midió cuando el error era menor al 0,1 %. Bajo esta condición el sistema se establece a los 0,665s, un valor bastante rápido dada la exigente condición impuesta. Finalmente se le impuso una referencia variada con el propósito de probar el rendimiento del sistema frente a cambios abruptos.

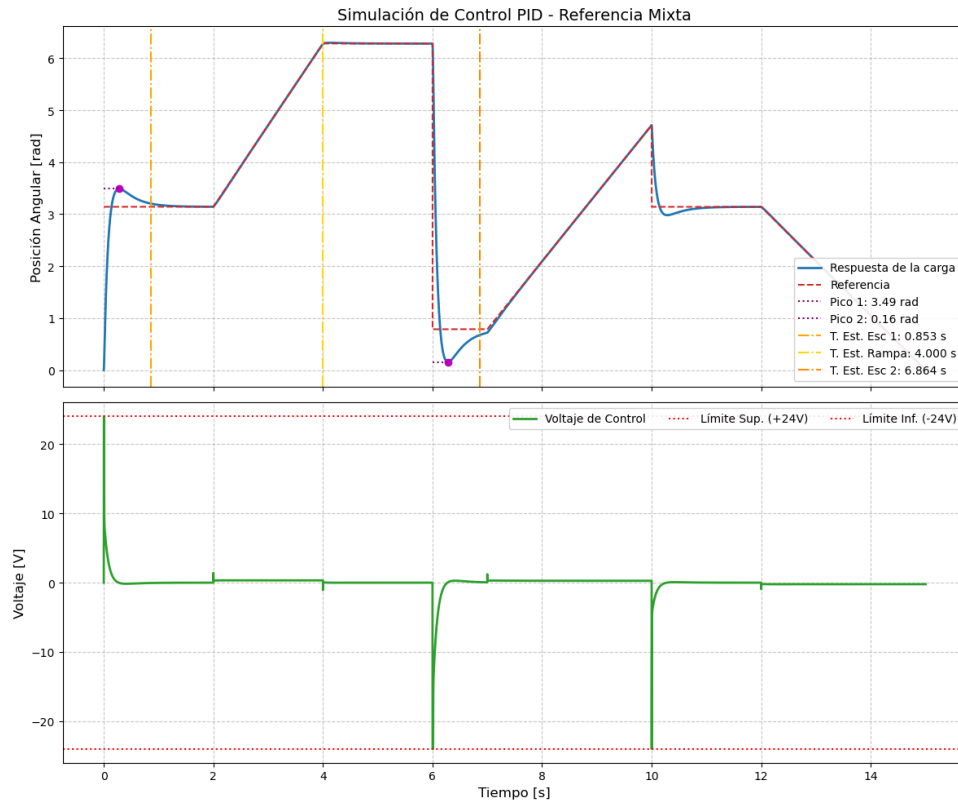


Figura 8: Respuesta del sistema frente a una referencia mixta.

Cuadro 3: Métricas de Desempeño

Tramo	Tipo	t_s (s)	M_p (%)	e_{ss} (rad)	V_{max} (V)
0s - 2s	Escalón	0.853	11.02	0.001	24.000
2s - 6s	Rampa	0.000	—	0.000	1.400
6s - 7s	Escalón	0.864	11.44	0.068	24.000
7s - 10s	Rampa	0.000	—	0.000	1.220
10s - 12s	Escalón	0.837	10.26	0.001	24.000

En la Figura 8 y en la Tabla 3 se puede ver que nuevamente los tiempos de establecimiento son menores al segundo y se mantienen alrededor de los 0,850s y el sobrepico se mantiene dentro de lo esperado. Algo a destacar es el error en estado estacionario luego del escalón entre 6s y 7s que deja un error en estado estacionario de 0,068 rads, esto corresponde a un error menor al 2% que es lo que se consideró como tiempo de establecimiento, por ende, aunque parezca un valor elevado comparado al error que se obtiene cuando le damos más tiempo al sistema de estabilizarse, este sigue siendo un error mínimo.

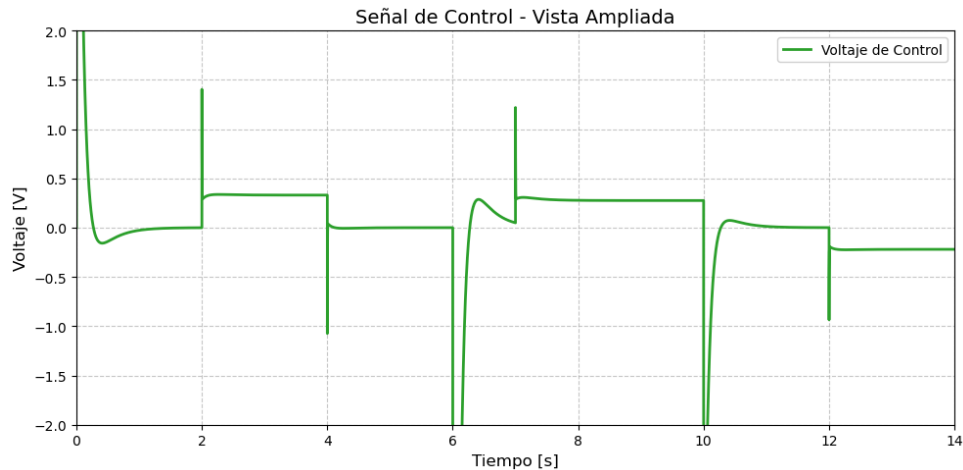


Figura 9: Zoom a la señal de control.

Otro detalle es que pareciera que en las rampas el sistema no inyecta voltaje; en realidad lo que hace es inyectar una cantidad relativamente grande al principio de la rampa para vencer la inercia, luego se mantiene a valores muy chicos hasta llegar al final donde inyecta una fuerza contraria para vencer la inercia y detener el eje. Esto puede verse con detalle en la Figura 9. Un problema que surgió durante la simulación fue el kick derivativo que producía el uso de escalones reales, dependiendo del tiempo usado para la simulación el valor de la derivada se hacía cada vez más grande, teniendo en cuenta que se partía de cero la derivada del error $\frac{de(t)}{dt}$ que en la simulación calculamos como

$$\frac{de(t)}{dt} \approx \frac{error_{actual} - error_{previo}}{dt}$$

y según el valor elegido para dt daba números más grandes a más chico era el intervalo de simulación, esto hacía que en los escalones el controlador exija cantidades elevadas de voltaje como $6,6kV$ o $5,1kV$. Esta exigencia del controlador caía inmediatamente a valores normales luego del primer kick, sin embargo se decidió evitar este primer golpe poniéndole una saturación al sistema para que se quedara obligatoriamente entre $-24V$ y $24V$. Para aplicar esta saturación simplemente se añadió al código una revisión del valor del voltaje.

4. Conclusión

1. Se verificó que el sistema planteado cumple con los requisitos impuestos previos al desarrollo del trabajo.
2. El tiempo de establecimiento, sobrepico y error en estado estacionario se mantuvieron en valores normales.
3. Se solucionó eficazmente el problema del kick derivativo mediante el uso de una saturación.
4. Se utilizó el metodo de Runge-Kutta para la aproximación al comportamiento real del sistema.

5. Fuentes

Referencias

- [1] Ogata, K. (2010). *Ingeniería de control moderna* (5ta ed.). Pearson Educación.

- [2] Kuo, B. C. (1996). *Sistemas de control automático* (7ma ed.). Prentice Hall Hispanoamericana.
- [3] maxon group. (s.f.). *RE 25, Ø 25 mm, Graphite Brushes, 20 Watt*. Recuperado el 3 de junio de 2026, de <https://www.maxongroup.com/maxon/view/product/motor/dcmotor/re/re25/118745>
- [4] Omron Automation and Safety. (s.f.). *E6B2-CWZ6C 1024P/R 0.5M - Rotary Encoders*. DigiKey. Recuperado el 3 de junio de 2026, de <https://www.digikey.com/en/products/detail/omron-automation-and-safety/E6B2-CWZ6C-1024P-R-0-5M/4079097>